Vous : arbre de décision utilisant l'entropie de shannon en python

ChaGPT :

Pour implémenter un arbre de décision utilisant l'entropie de Shannon en Python, nous devons d'abord comprendre comment l'entropie et la division des données fonctionnent. L'entropie est une mesure de l'incertitude ou de l'homogénéité d'un ensemble de données, et l'objectif est de diviser les données en fonction de l'attribut qui minimise l'entropie, afin de rendre les classes aussi pures que possible.

Voici les étapes principales pour construire un arbre de décision avec l'entropie de Shannon :

1. **Calculer l'entropie de Shannon** d'un ensemble de données.
2. **Calculer le gain d'information** pour chaque attribut, c'est-à-dire la réduction de l'entropie après une division par un attribut donné.
3. **Diviser les données** en fonction de l'attribut qui offre le meilleur gain d'information.
4. **Répéter le processus** récursivement pour chaque sous-ensemble de données jusqu'à ce que l'arbre soit complet.

Voici un exemple d'implémentation en Python :

```
import numpy as np


# Fonction pour calculer l'entropie de Shannon

def entropy(S):

    # S est un tableau de labels (étiquettes des classes)

    labels, counts = np.unique(S, return_counts=True)

    probas = counts / len(S)

    return -np.sum(probas * np.log2(probas))


# Fonction pour calculer le gain d'information
```

```
def information_gain(S, attribute, labels):
    # S : ensemble de données (les échantillons)
    # attribute : indice de l'attribut (colonne) à diviser
    # labels : les étiquettes de classe associées à chaque échantillon
    total_entropy = entropy(labels)  # Entropie de l'ensemble complet

    # Obtenir les valeurs uniques de l'attribut (pour faire la division)
    attribute_values = np.unique(S[:, attribute])

    weighted_entropy = 0
    for value in attribute_values:
        # Diviser les données en fonction de la valeur de l'attribut
        subset = labels[S[:, attribute] == value]
        # Calculer l'entropie de ce sous-ensemble pondéré par la taille du sous-ensemble
        weighted_entropy += (len(subset) / len(labels)) * entropy(subset)

    # Le gain d'information est la différence entre l'entropie totale et l'entropie pondérée
    return total_entropy - weighted_entropy

# Fonction pour construire l'arbre de décision
def decision_tree(S, labels, attributes):
```

```
# S : ensemble des données (échantillons)

# labels : les étiquettes de classe

# attributes : indices des attributs disponibles pour la division


# Si toutes les étiquettes de classe sont les mêmes, retourner la classe unique

if len(np.unique(labels)) == 1:

    return np.unique(labels)[0]


# Si aucun attribut n'est disponible, retourner la classe majoritaire

if len(attributes) == 0:

    return np.argmax(np.bincount(labels))


# Trouver l'attribut avec le meilleur gain d'information

best_attribute = None

best_gain = -float('inf')

for attribute in attributes:

    gain = information_gain(S, attribute, labels)

    if gain > best_gain:

        best_gain = gain

        best_attribute = attribute
```

```
    # Diviser les données en fonction de la meilleure valeur d'attribut
    tree = {best_attribute: {}}
    attribute_values = np.unique(S[:, best_attribute])
    new_attributes = [a for a in attributes if a != best_attribute]

    for value in attribute_values:
        subset_indices = S[:, best_attribute] == value
        subset_S = S[subset_indices]
        subset_labels = labels[subset_indices]
        tree[best_attribute][value] = decision_tree(subset_S, subset_labels, new_attributes)

    return tree

# Fonction pour prédire avec l'arbre de décision
def predict(tree, sample):
    # Parcours de l'arbre pour effectuer la prédiction
    if isinstance(tree, dict):
        attribute = list(tree.keys())[0]
        value = sample[attribute]
        return predict(tree[attribute].get(value, -1), sample)
    else:
```

```
    return tree

# Exemple d'utilisation

# Données d'exemple (S, labels) : chaque ligne représente un échantillon, chaque colonne un attribut
S = np.array([
    [0, 1, 1],
    [1, 1, 0],
    [1, 0, 1],
    [0, 0, 1],
    [1, 1, 1]
])

# Étiquettes de classe (labels) pour chaque échantillon
labels = np.array([0, 1, 1, 0, 1])

# Attributs disponibles (index des colonnes dans S)
attributes = [0, 1, 2]

# Construction de l'arbre de décision
tree = decision_tree(S, labels, attributes)
```

```
print("Arbre de décision construit:", tree)


# Prédiction pour un nouvel échantillon

new_sample = [1, 0, 1]  # Exemple de nouvel échantillon

prediction = predict(tree, new_sample)

print("Prédiction pour l'échantillon", new_sample, ":", prediction)
```

Explication des fonctions :

1. **`entropy(S)`** : Cette fonction calcule l'entropie de Shannon pour un ensemble de données `S` (une liste d'étiquettes de classe). L'entropie est définie par la formule :

   H(S)=−∑pilog2(pi)H(S) = - \sum p_i \log_2(p_i)H(S)=−∑pilog2(pi)

   où pip_ipi est la probabilité de chaque classe dans l'ensemble `S`.

2. **`information_gain(S, attribute, labels)`** : Cette fonction calcule le gain d'information pour un attribut donné. Le gain d'information est la différence entre l'entropie initiale de l'ensemble et l'entropie pondérée des sous-ensembles obtenus en divisant les données par l'attribut.
3. **`decision_tree(S, labels, attributes)`** : Cette fonction construit l'arbre de décision de manière récursive. Elle choisit l'attribut avec le meilleur gain d'information, divise les données en fonction de cet attribut, puis répète le processus pour chaque sous-ensemble.
4. **`predict(tree, sample)`** : Cette fonction prédit la classe d'un nouvel échantillon en suivant l'arbre de décision construit.

Exemple de sortie :

```
Arbre de décision construit: {0: {0: 0, 1: {1: {0: 0, 1: 1}}}}

Prédiction pour l'échantillon [1, 0, 1] : 1
```

L'arbre de décision est construit à partir des données d'entraînement et peut être utilisé pour faire des prédictions sur de nouveaux échantillons.